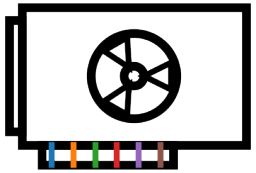
Neural Network Accelerator

Team 11
Kai Segimoto, Shang Wang

Importance of Neural Network Accelerator

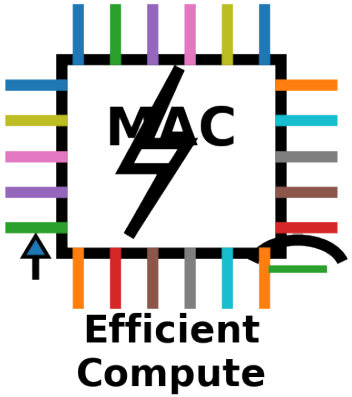




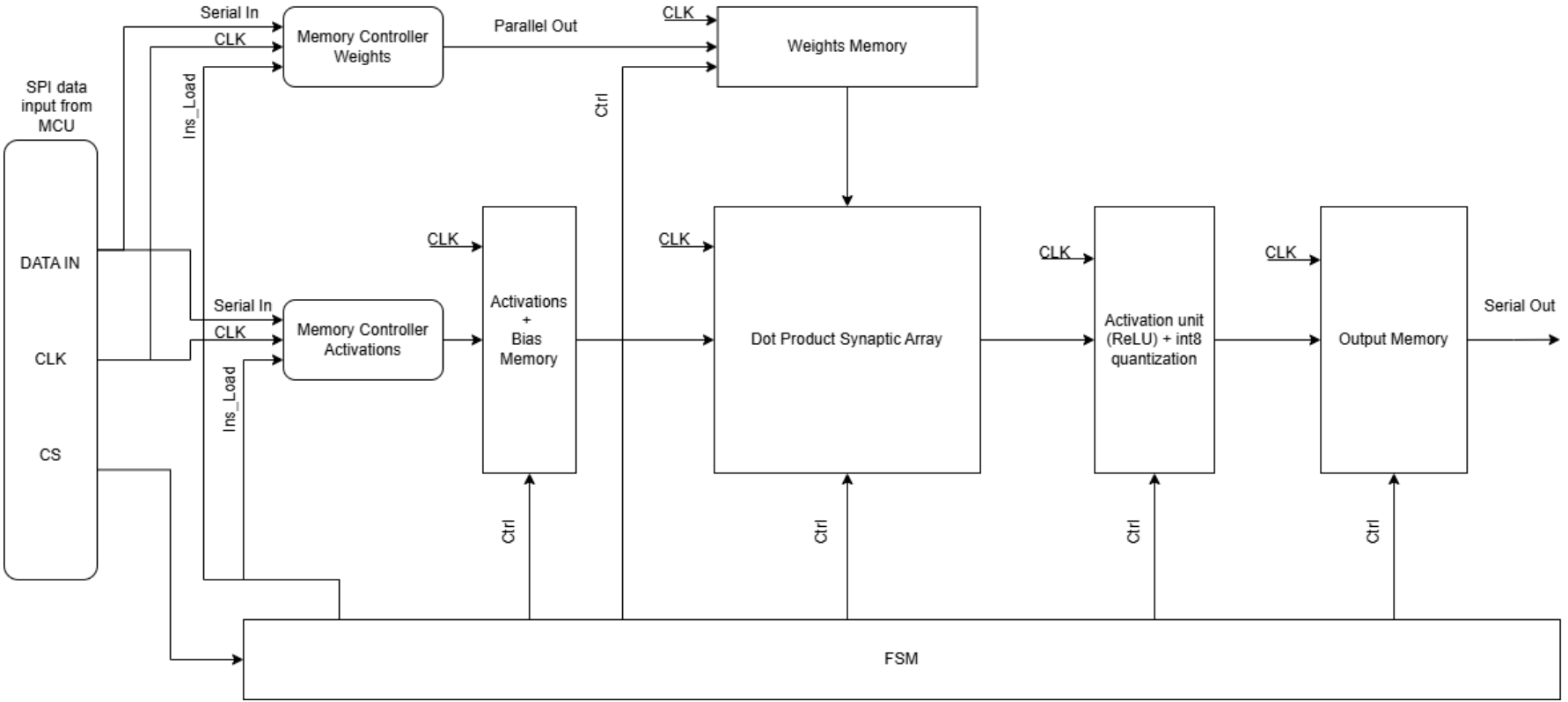
- As the demand for AI and machine learning grows, the need for faster, more efficient computing of models such as neural networks becomes increasingly important.
- Neural Networks and machine learning have various edge applications such as real-time image classification, keyword spotting, or anomaly detection rely on fast, low-power inference under tight latency and energy constraints.
- By implementing a neural networks on a hardware level including matrix multiplication and activation functions/quantization, performance factors such as latency, power consumption, and area can be greatly optimized.

Design Goals

- Design an **8-bit 4×4 matrix multiplier engine** (MAC-based systolic array) with a complete **FSM-controlled datapath**.
- Implement **separate SRAM blocks** for **weights** and **activations/bias**, including **row/column addressing, precharge, column mux, sense amp, and write driver** behavior at the block level.
- Implement Matrix multiplication using a **systolic array architecture**, which uses parallelism for highly increased throughput
- Add an **activation stage (ReLU) + int8 normalization**, then store results in an **output memory** with serial I/O interface support.
- Verify functionality with **RTL + testbenches**, then complete **synthesis and physical flow** (Cadence **liberate** characterization, **Genus** synthesis, **Innovus** P&R)

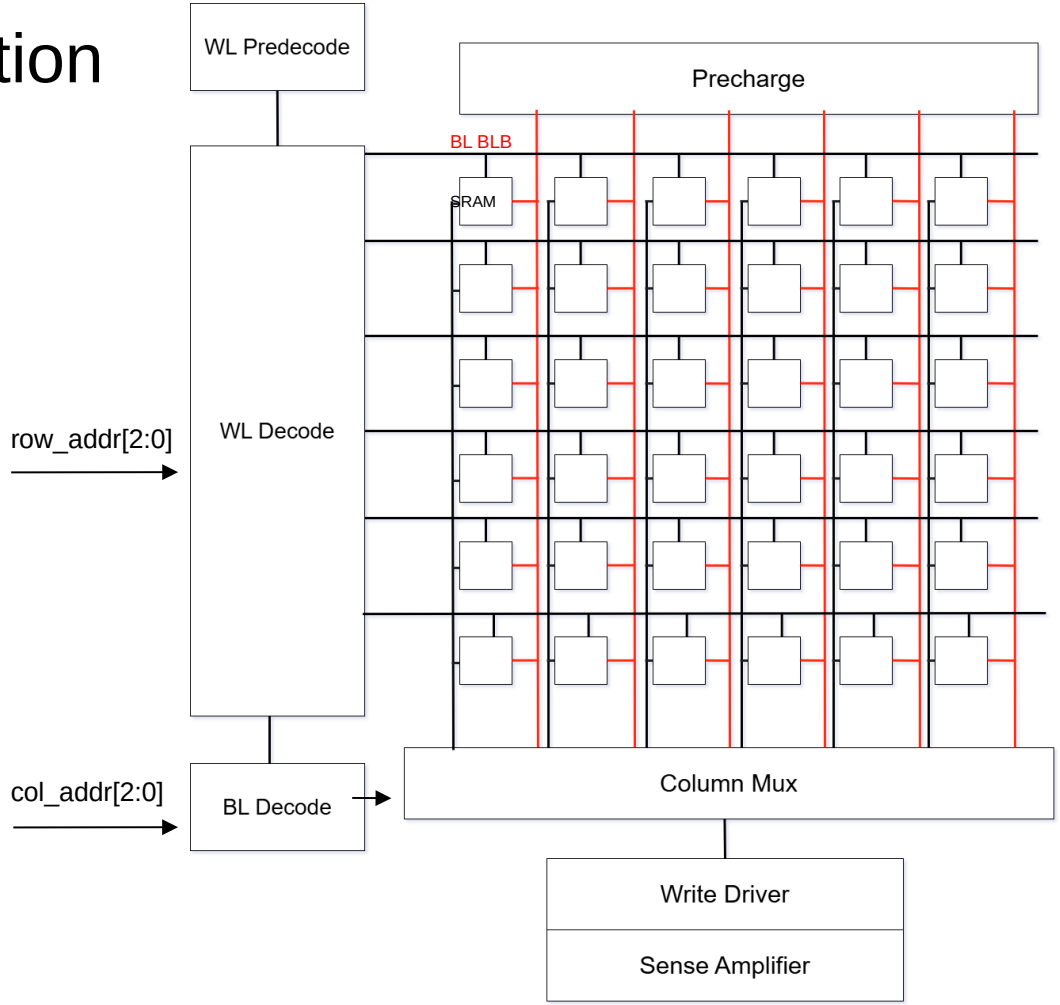


Block Diagram



Memory Array Operation

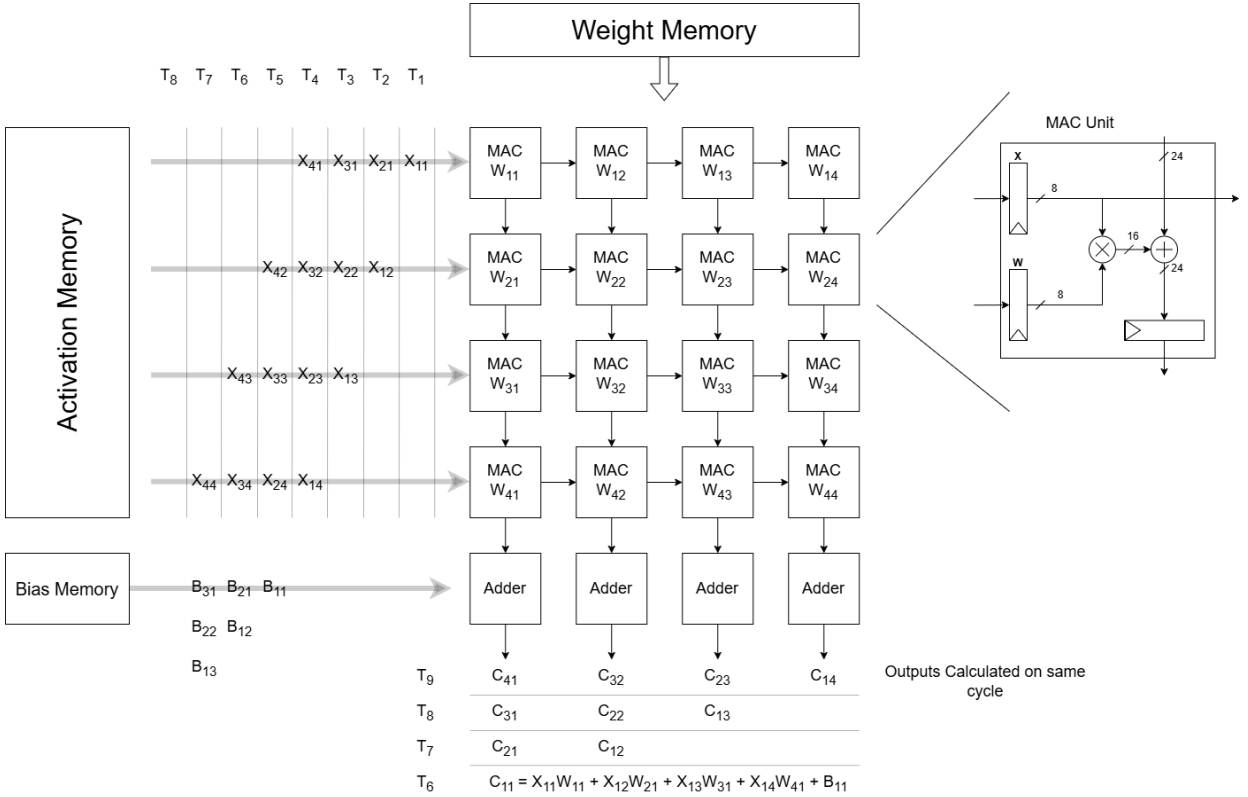
- Each block is an 8 bit SRAM
- The controller receives row_addr[2:0], col_addr[2:0], and read/write control signals from FSM.
- Before every read access, the precharge circuit equalizes and precharges each BL/BLB pair to a known level.
- The WL predecode stage converts the row address into intermediate decode signals
- The WL decode asserts exactly one wordline, enabling all cells in the selected row to connect to their BL/BLB pairs.
- The sense amplifier resolves and amplifies the BL/BLB differential to produce a stable digital output word.
- After the operation, the wordline is deasserted and the array returns to the precharge state for the next access.



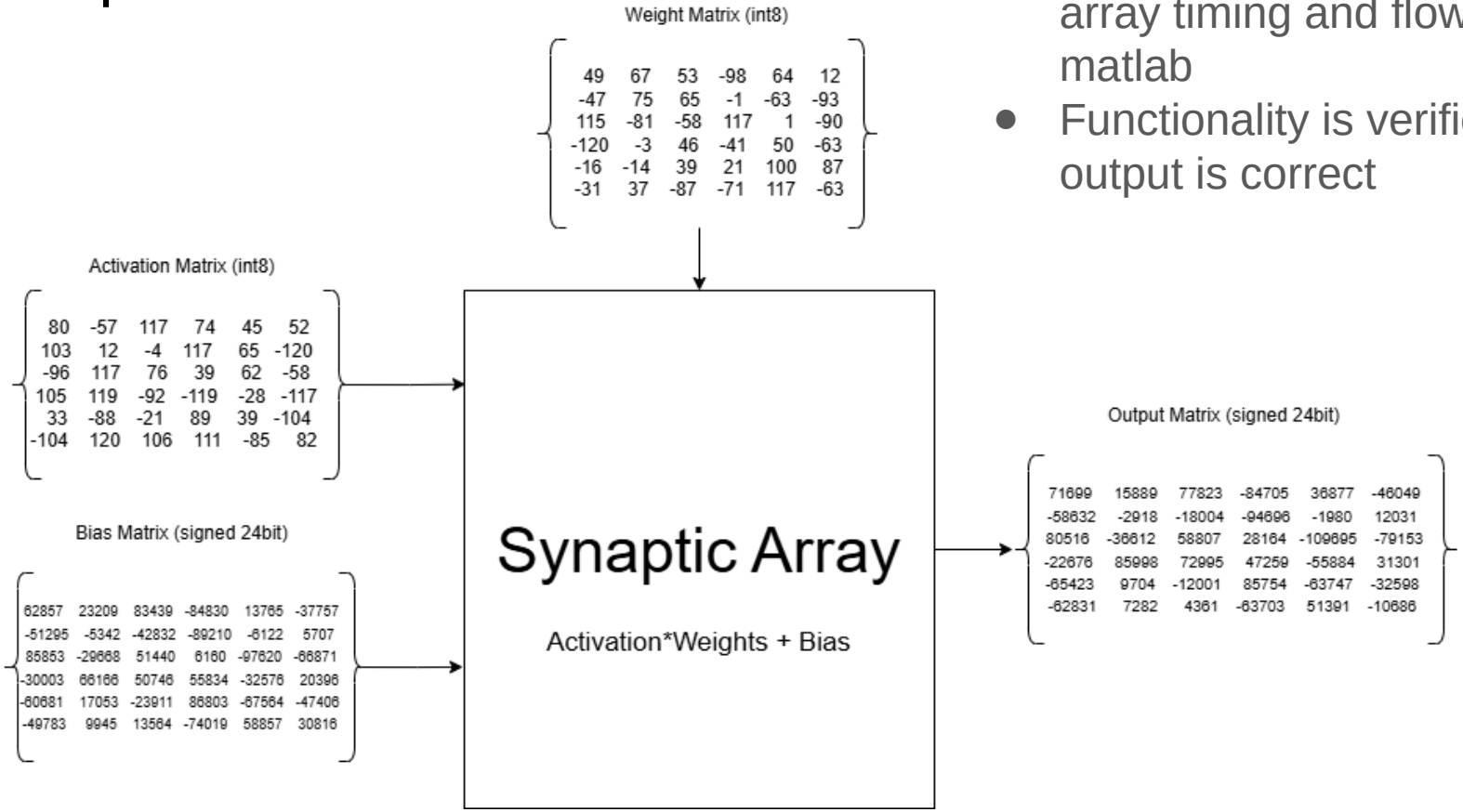
Dot Product Systolic Array Operation

- Dot product will be performed using a Systolic Array architecture for high efficiency data flow
- Weights are loaded into each MAC unit before computation
- Input activations flow to the right and partial sums flow down each clock cycle for data reuse and parallelism
- Increased throughput compared to individual cell calculations

- Inputs will be controlled by external control logic block



Dot product Simulation



- Script implementing systolic array timing and flow created in matlab
- Functionality is verified as output is correct

Timeline

Task	Start Date	End Date	Assigned to	Description
Design Confirmation	2.11	2.15	Both	Determine overall architecture,design specifications,and performance goals before starting
Verilog Coding and Simulation for Memory Array	2.15	2.20	Shang	Implement the memory controller RTL, write a testbench, and run simulation to verify control/timing via waveforms
Verilog Coding and Simulation for Memory Controller	2.21	2.28	Shang	Implement the memory array RTL, write a testbench, and run simulation to verify correctness
Verilog Coding and Simulation for Matrix multiplier	2.15	2.22	Kai	Implement the matrix multiplier RTL, write a testbench, and simulate to validate outputs and latency against theoretical value
Verilog Coding and Simulation for Activation unit	2.23	2.28	Kai	Implement the activation RTL, write a testbench, and simulate to verify mapping and overflow behavior
Verilog Coding and Simulation for FSM	3.1	3.9	Both	Implement the top-level FSM to coordinate modules, write an integration testbench, and simulate to verify end-to-end sequencing and state transitions.

Task	Start Date	End Date	Assigned to	Description
Verilog Coding and Simulation for Top Level Integration	3.10	3.16	Both	Integrate all RTL modules at the top level, connect interfaces, and run simulations to confirm signal connectivity and control flow.
Create SRAM Macro. Schematic + Layout	3.12	3.25	Kai	Create SRAM macro to be used for verilog code, generate .lib and .lef files.
Make and characterize standard cell library in Cadence	3.17	3.25	Shang	Create the **standard cells** and characterize them using Cadence Liberate to generate .lib models.
Synthesize verilog, port to Cadence	3.26	4.3	Both	Set up and run synthesis to produce the **gate-level netlist** and required **library/PDK files** for the Cadence digital flow.
Run and verify Innovus flow layout. Import DEF file into virtuoso. Fix DRC, LVS	4.4	4.12	Both	fix **DRC/LVS**: Run Innovus P&R, check timing/DRC/LVS, export DEF, import into Virtuoso, and resolve DRC/LVS/antenna issues.
Final Confirmation and Tape Out	4.13	4.20	Both	Perform clean DRC/LVS and required post-layout checks and generate the final deliverables for tape-out submission.